

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ_____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА _____СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Дисциплина: Парадигмы и конструкции языков программирования

Отчёт по лабораторной работе № 2

«Объектно-ориентированные возможности языка Python»

Выполнил: студент группы ИУ5-31Б _____ Синявский С.Д.

Проверил: _____ _____

2026 г.

1. Описание задания

Цель лабораторной работы: изучение объектно-ориентированных возможностей языка Python.

Вариант: N = 25.

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием pip.

2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.

3. Все файлы проекта (кроме основного файла main.py) должны располагаться в пакете lab_python_oop.

4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета lab_python_oop.

5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.

6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.

7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.

8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа math.pi из модуля math.

9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:

• определите метод "repr", который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод format;

• название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.

10. В корневом каталоге проекта создайте файл main.py для тестирования Ваших классов. Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):

• прямоугольник синего цвета шириной N и высотой N;

• круг зеленого цвета радиусом N;

• квадрат красного цвета со стороной N;

• также вызовите один из методов внешнего пакета, установленного с использованием pip.

11. Дополнительное задание. Протестируйте корректность работы Вашей программы с помощью модульного теста.

2. Текст программы

2.1. Файл main.py (основной файл программы)

```
import numpy

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

N = 25


def main():
    rectangle = Rectangle(N, N, 'синего')
    circle = Circle(N, 'зеленого')
    square = Square(N, 'красного')
    print(rectangle)
    print(circle)
    print(square)
    # вызов метода внешнего пакета numpy
    total_area = numpy.sum([rectangle.area(), circle.area(), square.area()])
    print('Суммарная площадь фигур (numpy.sum): {:.2f}'.format(total_area))


if __name__ == '__main__':
    main()
```

2.2. Файл lab_python_oop/figure.py (абстрактный класс «Геометрическая фигура»)

```
from abc import ABC, abstractmethod


class Figure(ABC):

    @abstractmethod
    def area(self):
        pass
```

2.3. Файл lab_python_oop/color.py (класс «Цвет фигуры»)

```
class FigureColor:

    def __init__(self, color):
        self._color = color

    # свойство: чтение и запись цвета
    @property
    def color(self):
        return self._color

    @color.setter
    def color(self, value):
        self._color = value
```

2.4. Файл lab_python_oop/rectangle.py (класс «Прямоугольник»)

```
from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor


class Rectangle(Figure):
    # название фигуры - поле данных класса
    FIGURE_TYPE = 'Прямоугольник'

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        # цвет хранится в объекте класса «Цвет фигуры»
        self.figure_color = FigureColor(color)

    def area(self):
        return self.width * self.height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {} площадью {}.'.format(
            self.get_figure_type(),
            self.figure_color.color,
            self.width,
            self.height,
            self.area()
        )
```

2.5. Файл lab_python_oop/circle.py (класс «Круг»)

```
import math

from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor


class Circle(Figure):
    FIGURE_TYPE = 'Круг'

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, radius, color):
        self.radius = radius
        self.figure_color = FigureColor(color)

    def area(self):
        return math.pi * self.radius ** 2

    def __repr__(self):
        return '{} {} цвета радиусом {} площадью {:.2f}.'.format(
            self.get_figure_type(),
            self.figure_color.color,
            self.radius,
            self.area()
        )
```

2.6. Файл lab_python_oop/square.py (класс «Квадрат»)

```
from lab_python_oop.rectangle import Rectangle


class Square(Rectangle):
    FIGURE_TYPE = 'Квадрат'

    def __init__(self, side, color):
        self.side = side
        # квадрат - прямоугольник с равными сторонами
        super().__init__(side, side, color)

    def __repr__(self):
        return '{} {} цвета со стороной {} площадью {}.'.format(
            self.get_figure_type(),
            self.figure_color.color,
            self.side,
            self.area()
        )
```

2.7. Файл lab_python_oop/test_figures.py (модульный тест)

```
import math
import unittest

from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square


class TestFigures(unittest.TestCase):

    def test_figure_is_abstract(self):
        # объект абстрактного класса создать нельзя
        with self.assertRaises(TypeError):
            Figure()

    def test_color_property(self):
        figure_color = FigureColor('синего')
        figure_color.color = 'желтого'
        self.assertEqual(figure_color.color, 'желтого')

    def test_rectangle(self):
        rectangle = Rectangle(25, 25, 'синего')
        self.assertEqual(rectangle.area(), 625)
        self.assertEqual(Rectangle.get_figure_type(), 'Прямоугольник')
        self.assertEqual(repr(rectangle),
                         'Прямоугольник синего цвета шириной 25 и высотой 25 площадью 625.')

    def test_circle(self):
        circle = Circle(25, 'зеленого')
        self.assertAlmostEqual(circle.area(), math.pi * 625)
        self.assertEqual(Circle.get_figure_type(), 'Круг')
        self.assertEqual(repr(circle),
                         'Круг зеленого цвета радиусом 25 площадью 1963.50.')

    def test_square(self):
        square = Square(25, 'красного')
        self.assertIsInstance(square, Rectangle)
        self.assertEqual(square.area(), 625)
        self.assertEqual(Square.get_figure_type(), 'Квадрат')
        self.assertEqual(repr(square),
                         'Квадрат красного цвета со стороной 25 площадью 625.')


if __name__ == '__main__':
    unittest.main()
```

3. Экранные формы с примерами выполнения программы

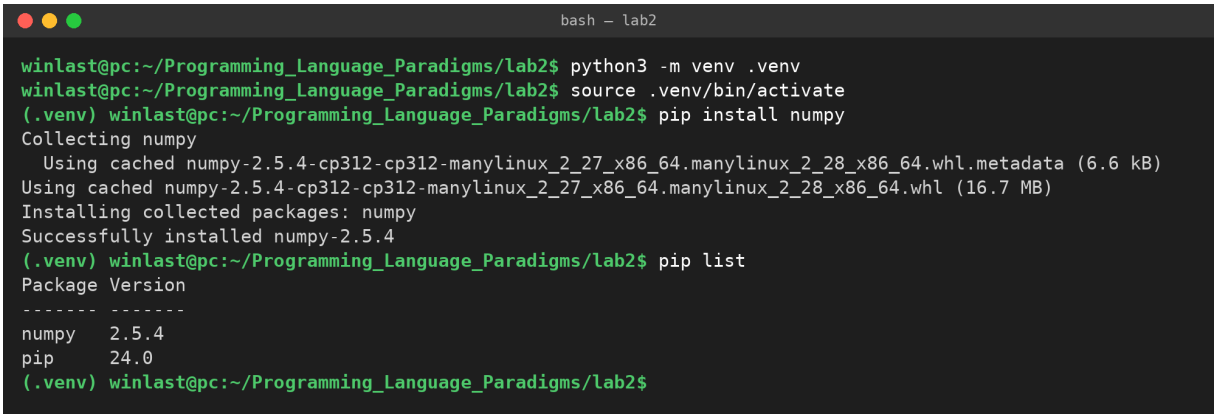


Рисунок 1 — Создание виртуального окружения и установка пакета numpy

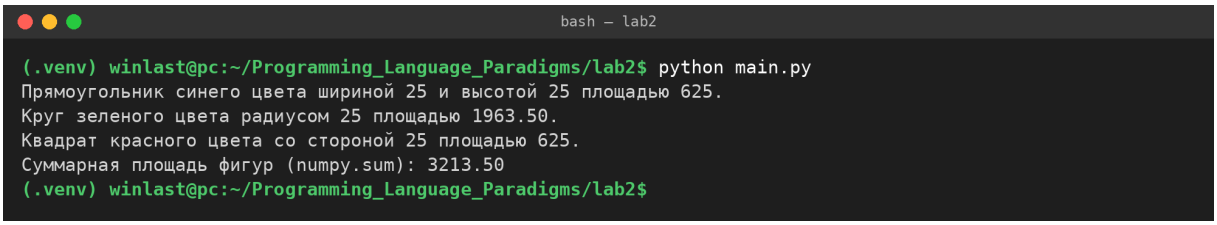


Рисунок 2 — Выполнение программы main.py

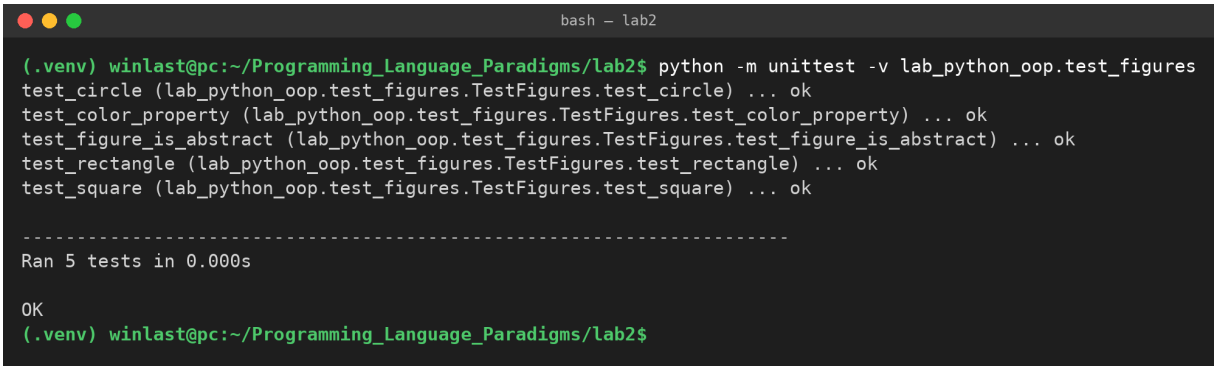


Рисунок 3 — Выполнение модульного теста